

DBMS LAB

Leetcode SQL Problems

Assignment - 4

Submitted by: Ganji Satwika

Roll No: 24MAB0A09

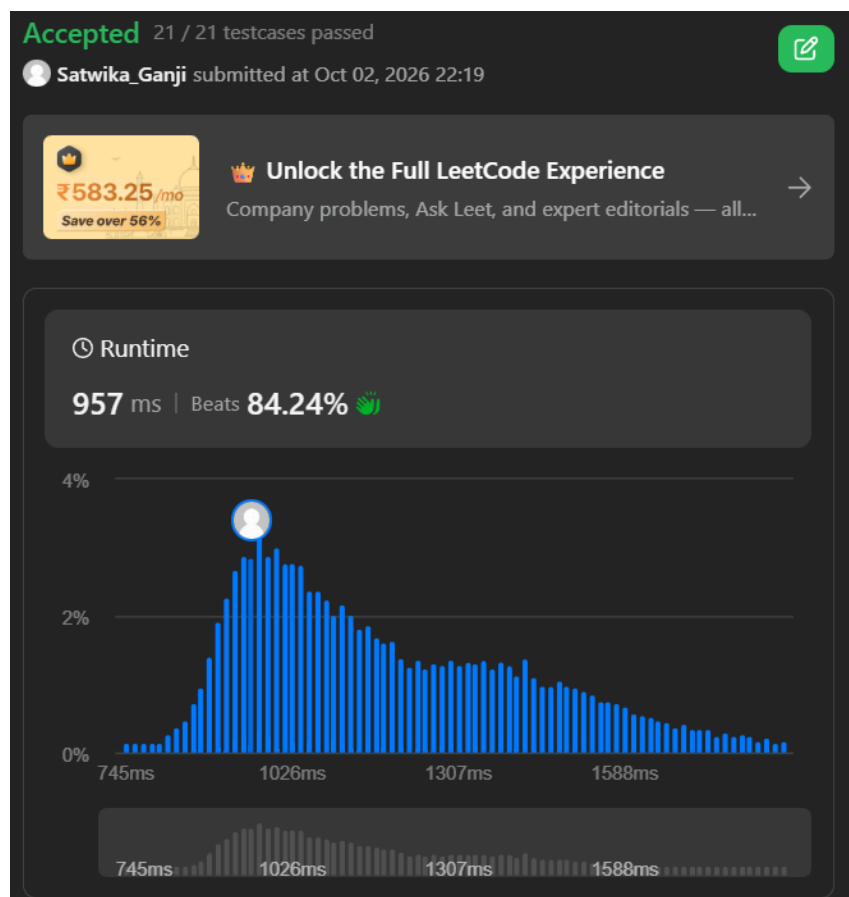
Submitted to: Dr. Vikas Srivastava

Course: DBMS Lab

Date: 2 October 2026

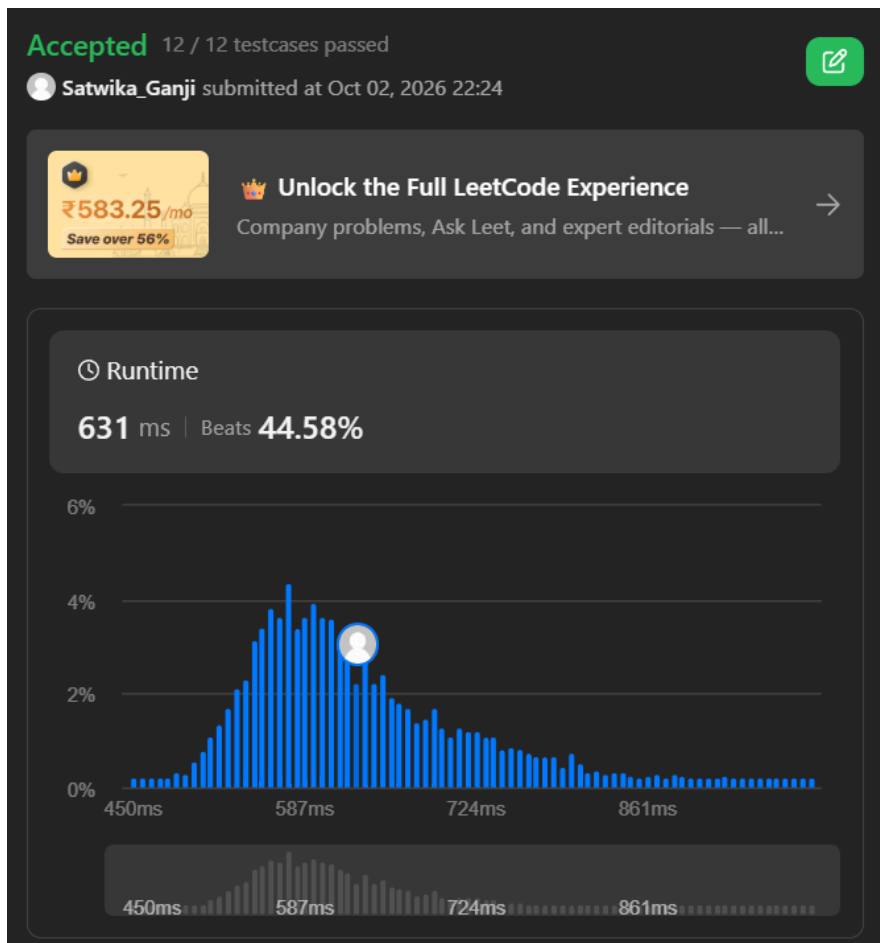
1. Leetcode 185 - Department Top Three Salaries

```
SELECT Department, Employee, Salary
FROM (
    SELECT d.name AS Department,
           e.name AS Employee,
           e.salary AS Salary,
           DENSE_RANK() OVER (PARTITION BY d.id ORDER BY e.salary DESC) AS rnk
    FROM Employee e
    JOIN Department d ON e.departmentId = d.id
) t
WHERE rnk <= 3;
```



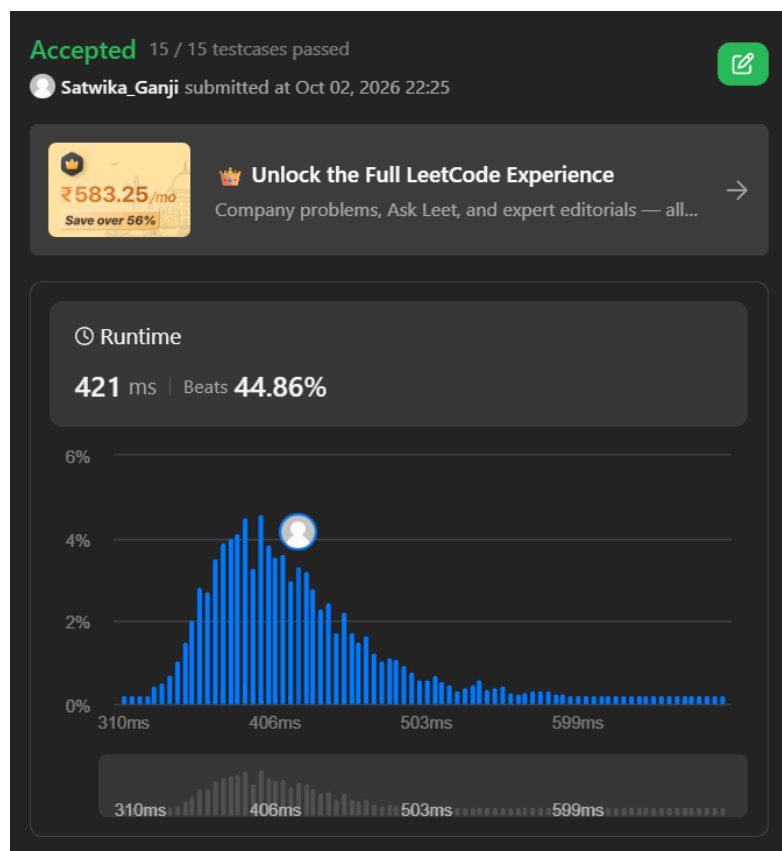
2. LeetCode 262, Trips and Users

```
SELECT t.request_at AS Day,  
       ROUND(AVG(t.status != 'completed'), 2) AS `Cancellation Rate`  
FROM Trips t  
JOIN Users c ON t.client_id = c.users_id AND c.banned = 'No'  
JOIN Users d ON t.driver_id = d.users_id AND d.banned = 'No'  
WHERE t.request_at BETWEEN '2013-10-01' AND '2013-10-03'  
GROUP BY t.request_at;
```



3. LeetCode601, Human Traffic of Stadium

```
SELECT id, visit_date, people
FROM (
  SELECT *,
    COUNT(*) OVER (PARTITION BY id - rn) AS cnt
  FROM (
    SELECT *,
      ROW_NUMBER() OVER (ORDER BY id) AS rn
    FROM Stadium
    WHERE people >= 100
  ) t
) s
WHERE cnt >= 3
ORDER BY visit_date;
```



4. LeetCode3374, First Letter Capitalization II

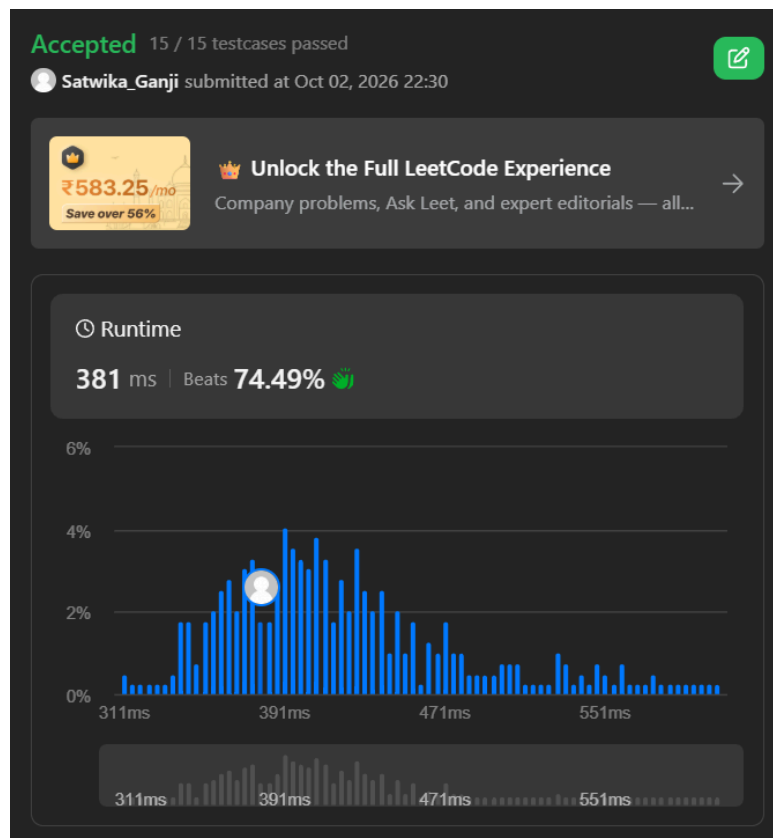
Write your MySQL query statement below

```
WITH RECURSIVE w AS (  
    SELECT content_id, content_text, 1 AS n,  
           CHAR_LENGTH(content_text) - CHAR_LENGTH(REPLACE(content_text, ' ', '')) + 1  
    AS cnt,  
           SUBSTRING_INDEX(content_text, ' ', 1) AS word  
    FROM user_content  
    UNION ALL  
    SELECT content_id, content_text, n + 1, cnt,  
           SUBSTRING_INDEX(SUBSTRING_INDEX(content_text, ' ', n + 1), ' ', -1)  
    FROM w  
    WHERE n < cnt  
)  
p AS (  
    SELECT content_id, n, 1 AS m, word,  
           IF(word REGEXP '^[A-Za-z]+(-[A-Za-z]+)+$',  
              CHAR_LENGTH(word) - CHAR_LENGTH(REPLACE(word, '-', '')) + 1, 1) AS pc  
    FROM w  
    UNION ALL  
    SELECT content_id, n, m + 1, word, pc  
    FROM p  
    WHERE m < pc  
)  
c AS (  
    SELECT content_id, n,  
           GROUP_CONCAT(  
               IF(word REGEXP '^[A-Za-z]',  
                  CONCAT(UPPER(LEFT(x.part, 1)), LOWER(SUBSTRING(x.part, 2))),  
                  x.part)  
               ORDER BY m SEPARATOR '-') AS cw  
    FROM (  
        SELECT content_id, n, m, word, pc  
        FROM p  
        WHERE m < pc  
    )
```

```

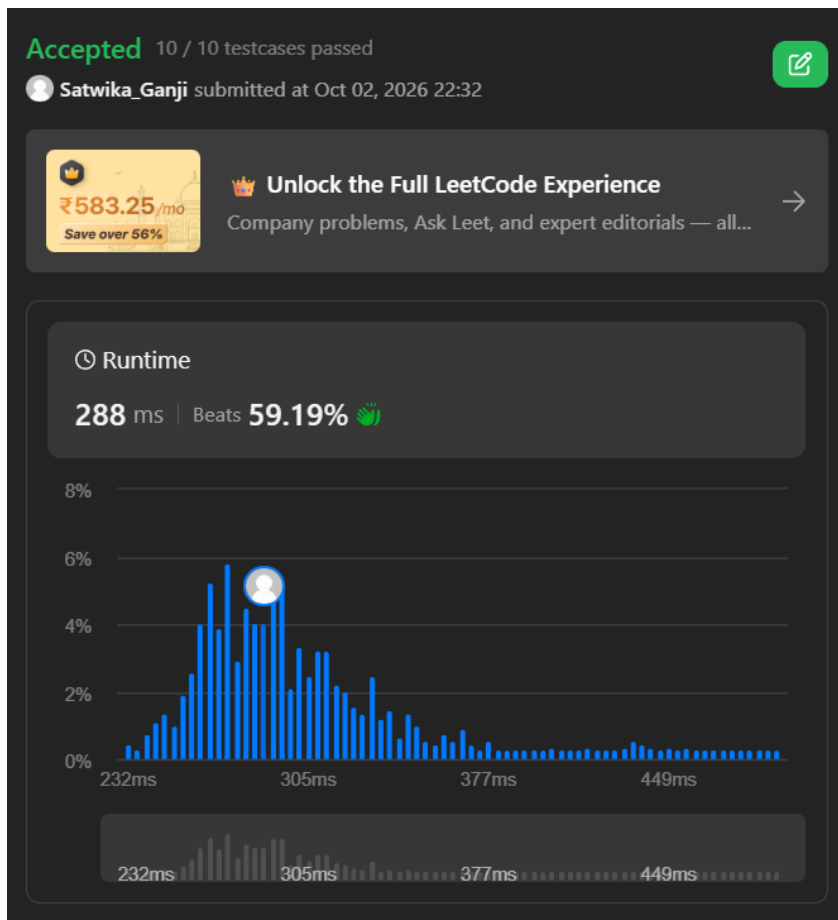
        SELECT content_id, n, m, word,
               IF(pc > 1, SUBSTRING_INDEX(SUBSTRING_INDEX(word, '-', m), '-', -1), word)
        AS part
        FROM p
    ) x
    GROUP BY content_id, n
)
SELECT u.content_id,
       u.content_text AS original_text,
       GROUP_CONCAT(c.cw ORDER BY c.n SEPARATOR ' ') AS converted_text
FROM user_content u
JOIN c ON u.content_id = c.content_id
GROUP BY u.content_id, u.content_text;

```



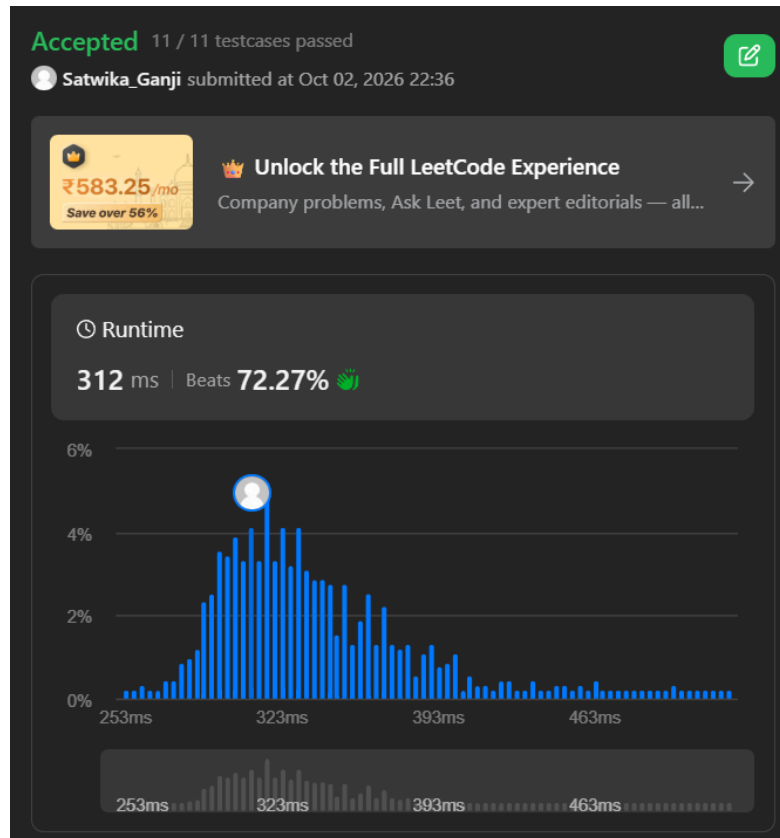
5. LeetCode 3451, Find Invalid IP Addresses

```
SELECT ip, COUNT(*) AS invalid_count
FROM logs
WHERE ip NOT REGEXP
'^(25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])([.]){3}(25[0-5]|2[0-4][0-9]|1[0-9]{2}|[1-9]?[0-9])$'
GROUP BY ip
ORDER BY invalid_count DESC, ip DESC;
```



6. LeetCode 3482, Analyze Organization Hierarchy

```
WITH RECURSIVE lv AS (  
    SELECT employee_id, 1 AS level  
    FROM Employees  
    WHERE manager_id IS NULL  
    UNION ALL  
    SELECT e.employee_id, lv.level + 1  
    FROM Employees e  
    JOIN lv ON e.manager_id = lv.employee_id  
)  
t AS (  
    SELECT employee_id AS root, employee_id AS id, salary  
    FROM Employees  
    UNION ALL  
    SELECT t.root, e.employee_id, e.salary  
    FROM t  
    JOIN Employees e ON e.manager_id = t.id  
)  
SELECT e.employee_id,  
       e.employee_name,  
       lv.level,  
       COUNT(*) - 1 AS team_size,  
       SUM(t.salary) AS budget  
FROM t  
JOIN Employees e ON e.employee_id = t.root  
JOIN lv ON lv.employee_id = t.root  
GROUP BY e.employee_id, e.employee_name, lv.level  
ORDER BY lv.level, budget DESC, e.employee_name;
```



7. LeetCode 3554, Find Category Recommendation Pairs

```
WITH uc AS (  
    SELECT DISTINCT p.user_id, i.category  
    FROM ProductPurchases p  
    JOIN ProductInfo i ON p.product_id = i.product_id  
)  
SELECT a.category AS category1,  
       b.category AS category2,  
       COUNT(*) AS customer_count  
FROM uc a  
JOIN uc b  
    ON a.user_id = b.user_id  
   AND a.category < b.category
```

GROUP BY a.category, b.category

HAVING COUNT(*) >= 3

ORDER BY customer_count DESC, category1, category2;

